

CSL GraphBuilder — Architectural Walk-through

Coroutines, Threads, Queues, and Pools in the Backend

1 Overview

CSL GraphBuilder is a biomedical knowledge-graph construction service. It ingests documents (or external sources such as Open Targets), extracts entities and relationships, embeds them with a domain-specific sentence-transformer (SapBERT), persists everything to Neo4j, and then runs a cascading verification pipeline before exposing the graph to a Next.js frontend.

The system is split into three deployable units:

- **Frontend** — Next.js dev server on port 3000 (`frontend/`).
- **Backend** — FastAPI/Uvicorn on port 8000 (`api/`).
- **Database** — Neo4j with native vector indexes for embedding-based retrieval.

The startup script that wires the first two together is `start.sh`; it cleans up listeners on ports 3000/8000 (filtered with `lsof -sTCP:LISTEN` so only servers, not browser client connections, are killed), activates the Python venv, and launches both processes.

2 Backend Layered Architecture

The Python backend follows a hexagonal layering. Within `src/graphbuilder/`:

- `domain/models/` — pure dataclasses (`KnowledgeGraph`, `GraphRelationship`, `DocumentChunk`). No I/O, no async.
- `core/` — domain services that don't own state: `processing/semantic_chunker.py`, `verification/cascading_verification/embedding.py`, `verification/llm_verifier.py`.
- `application/use_cases/` — orchestrators that compose domain services with infrastructure ports (`document_pipeline.py`, `relationship_verification.py`, `open_targets_ingestion.py`, ...).
- `infrastructure/` — adapters: Neo4j repository (`repositories/graph_repository.py`), embedding factory (`services/embedding_factory.py`), GPU pool (`services/gpu_embedding_pool.py`), LLM service (`services/llm_service.py`), config (`config/settings.py`).
- `cli/main.py` — Click CLI for offline batch processing.

The HTTP surface lives in `api/main.py` and the routers under `api/routers/` (`ingest.py`, `graph.py`, `verification.py`, `documents.py`, `curation.py`, `health.py`, `export.py`, `dev.py`). Routers are thin: they parse the request, call an application use-case, and shape the response.

3 The Three Concurrency Primitives

The backend deliberately mixes three forms of concurrency, each chosen for the workload it is best at. Understanding which primitive owns which job is the key to reading the codebase.

3.1 Coroutines (asyncio) — request and pipeline orchestration

Every router handler is `async`. The application layer is `async` all the way down: use-cases stage their work as `await`-able steps, with a `cancel_check` barrier between each stage so that long-running pipelines can be aborted cleanly when a user cancels a job. The canonical example is `DocumentPipelineUseCase` in `src/graphbuilder/application/use_cases/document_pipeline.py`, which executes:

1. `_stage_chunk` — semantic chunking
2. `_stage_entities` — entity extraction
3. `_stage_relationships` — relationship extraction
4. `_stage_persist` — save to Neo4j
5. `_stage_verify` — cascading verification

Coroutines own *orchestration*, not CPU-bound work. They are cheap to create, they preserve a single shared event loop, and they let the pipeline issue many concurrent network and database calls (e.g. Neo4j batch writes, LLM calls) without spawning OS threads.

Why this matters for the LLM call. `LLMVerifier.verify` in `src/graphbuilder/core/verification/llm.py` is `async` and awaits the underlying `llm_service.generate_text`. Many such verifications can be in flight on the same loop at once, limited only by the `VERIFY_PARALLEL_WORKERS` setting in `src/graphbuilder/infrastructure/config/settings.py`.

3.2 Threads — where blocking math happens

The event loop must *never* block. Two situations force us into threads:

1. **PyTorch / sentence-transformers inference.** `SentenceTransformer.encode` is a synchronous CUDA-or-CPU call that takes tens to hundreds of milliseconds. Calling it directly from an `async` handler freezes `/health` and every other in-flight request.
2. **Model warm-up and pool initialisation.** `SapBERT` is ~440 MB and takes 5–15 s to load. Doing this synchronously at FastAPI startup would delay the first request.

The threading lives in `src/graphbuilder/infrastructure/services/gpu_embedding_pool.py` and `src/graphbuilder/infrastructure/services/embedding_factory.py`:

- One *init thread* (`daemon=True`, `name="GPUEmbeddingPool-init"`) loads + warms a base model, measures per-worker GPU memory, then spawns the worker threads. The constructor returns immediately so FastAPI startup is not blocked.
- *N worker threads* (`name="GPUEmbeddingWorker-i"`), each owning its own `SentenceTransformer` replica and a dedicated `torch.cuda.Stream` so the GPU scheduler can interleave kernels instead of serialising on the default stream.
- On *CPU-only* machines the pool is never instantiated; a single `asyncio.Lock` plus the default `run_in_executor` thread are used (see `embed_async` / `embed_batch_async` in `embedding_factory.py`). The lock is required because PyTorch CPU inference on the same model is not safe under concurrent access; on CPU there is also nothing to gain from parallel encodes because MKL/OpenMP already saturates cores from a single call.

3.3 Queue — the GPU work-stealing pool

The single coupling between the asyncio world and the worker threads is a `queue.Queue` of `_Job` items inside `GPUEmbeddingPool`:

```
@dataclass
class _Job:
    payload: Union[str, List[str]]
    is_batch: bool
    batch_size: int
    future: asyncio.Future
    loop: asyncio.AbstractEventLoop
```

The async API (`embed_async`, `embed_batch_async`) creates an `asyncio.Future` on the calling loop, packages it into a `_Job`, drops it on the queue, and `await`s the future. Worker threads pull from the queue with a 0.5 s timeout (so they can notice the `stop_event` during shutdown), run the encode under their CUDA stream, and resolve the future with `loop.call_soon_threadsafe`—the result lands back on the event-loop thread cleanly, with no asyncio primitives touched off-loop.

The load-balancing policy is *whoever is free first grabs the next job*. This is the simplest correct algorithm and matches what the GPU actually wants: a busy worker does not get a new job, an idle one does. Per-worker counters (`jobs_in_flight`) are tracked under a `threading.Lock` for observability.

3.4 Pools — one model replica per worker

The pool’s worker count is sized once at start-up. After loading and warming a single model, peak GPU memory is read with `torch.cuda.max_memory_allocated` (this captures weights *plus* activations for a 32-batch forward pass), then:

$$n = \min\left(\text{max_workers}, \max\left(\text{min_workers}, \left\lfloor \frac{\text{free_bytes} \cdot \text{memory_fraction}}{\text{per_worker_bytes}} \right\rfloor\right)\right)$$

with defaults `min_workers=1`, `max_workers=8`, `memory_fraction=0.7`. Each of the resulting n workers loads its own `SentenceTransformer` replica into VRAM. The number is **static for the lifetime of the process**: there is no auto-scaler, and `shutdown()` only signals workers to stop, it does not resize.

Override knobs (read by `embedding_factory.py`):

- `EMBEDDING_GPU_WORKERS` — exact count, bypasses the formula above.
- `EMBEDDING_GPU_MIN_WORKERS` / `_MAX_WORKERS` — clamp range.
- `EMBEDDING_GPU_MEMORY_FRACTION` — fraction of free VRAM the pool may consume.
- `EMBEDDING_GPU_DEVICE` — target device, default `cuda:0`.

4 End-to-End Flow of an Embedding

The following walk-through traces a single embedding request from the chunker down to the GPU and back, illustrating how all four primitives collaborate.

1. A document arrives via `POST /documents` (router: `api/routers/documents.py`). FastAPI’s event loop schedules the handler as a coroutine.
2. The handler delegates to `DocumentPipelineUseCase.execute` (`src/graphbuilder/application/use_case/_stage_chunk` constructs a `SemanticChunker` and calls `await chunker.chunk(content, document.id)`).

3. Inside `SemanticChunker._embed` (`src/graphbuilder/core/processing/semantic_chunker.py`) the sentence list is passed to `embed_batch_async(sentences)`.
4. `embed_batch_async` in `embedding_factory.py` probes for a GPU pool. If present, it forwards to `GPUEmbeddingPool.embed_batch_async`; if not, it takes the per-loop `asyncio.Lock` and runs `embed_batch` in the default thread executor.
5. (*GPU path.*) The pool builds a `_Job` with the coroutine’s future and puts it on the worker queue. The coroutine `awaits` the future.
6. A free worker thread pops the job, switches to its dedicated `torch.cuda.Stream`, and runs `model.encode`.
7. The worker resolves the future via `loop.call_soon_threadsafe(future.set_result, result)`. The event loop’s I/O thread re-enters the awaiting coroutine on the next tick.
8. The chunker turns the returned `List[Optional[List[float]]]` into an (N, D) `np.ndarray`, runs the sentence-grouping algorithm, and returns `DocumentChunks` up the call stack.

The same pattern recurs for entity / relationship embeddings during `save_entities_batch` and `save_relationships_batch` in `src/graphbuilder/infrastructure/repositories/graph_repository.py`.

5 Verification Cascade

A second concurrency hot spot is verification, in `src/graphbuilder/core/verification/cascading.py`. Each relationship goes through up to three stages, in order of cost:

1. **Text-match** — substring/lexical, instant.
2. **Embedding** — cosine similarity against the relationship’s source chunk; uses the same `embed_batch_async` path.
3. **LLM** — structured-JSON prompt to `LLMVerifier` (`llm_verifier.py`), gated by `VERIFY_BATCH_SKIP_LLM` (default `true`, so Stage 3 is skipped during pipeline batches and runs only when the user manually triggers it from the `/verification` page).

The verification use-case (`application/use_cases/relationship_verification.py`) iterates relationships in coroutine form and dispatches the three stages sequentially per relationship; concurrency between relationships is controlled by the bounded `VERIFY_PARALLEL_WORKERS` semaphore in the pipeline orchestrator.

6 Configuration Surface

All concurrency-related knobs live in `src/graphbuilder/infrastructure/config/settings.py`:

Setting / env var	Default	Effect
<code>PROCESSING_MODE</code>	<code>batch</code>	Selects bulk vs single-doc paths
<code>VERIFY_ENABLED</code>	<code>true</code>	Master switch for the verify stage
<code>VERIFY_BATCH_SKIP_LLM</code>	<code>true</code>	Skip LLM stage in pipeline batches
<code>VERIFY_PARALLEL_WORKERS</code>	4	In-pipeline relationship concurrency
<code>EMBEDDING_GPU_WORKERS</code>	—	Force exact pool size
<code>EMBEDDING_GPU_MIN_WORKERS</code>	1	Pool sizing floor
<code>EMBEDDING_GPU_MAX_WORKERS</code>	8	Pool sizing ceiling
<code>EMBEDDING_GPU_MEMORY_FRACTION</code>	0.7	VRAM share for pool
<code>EMBEDDING_GPU_DEVICE</code>	<code>cuda:0</code>	Target CUDA device
<code>EMBEDDING_MODEL</code>	SapBERT	Sentence-transformer model name

7 File Index

Path	Role
<code>start.sh</code>	Process supervisor, port cleanup
<code>api/main.py</code>	FastAPI app + lifespan hooks
<code>api/routers/*.py</code>	HTTP routers (ingest, graph, verify, ...)
<code>application/use_cases/document_pipeline_async.py</code>	Async ingest pipeline
<code>application/use_cases/relationship_verification.py</code>	Verification orchestrator
<code>application/use_cases/open_targets_ingestion.py</code>	External source ingestion
<code>core/processing/semantic_chunker.py</code>	Sentence-aware async chunker
<code>core/verification/cascading.py</code>	Three-stage verifier
<code>core/verification/llm_verifier.py</code>	LLM JSON verifier
<code>infrastructure/services/embedding_fact_backend.py</code>	Backend-selection async API
<code>infrastructure/services/gpu_embedding_pool.py</code>	Threaded GPU pool + queue
<code>infrastructure/services/llm_service.py</code>	LLM client
<code>infrastructure/repositories/graph_repository.py</code>	Neighborhood persistence
<code>infrastructure/config/settings.py</code>	Env-driven configuration

8 Take-aways

- **Coroutines own orchestration.** Every router and use-case is `async`; the event loop is the single coordination point.
- **Threads own blocking math.** GPU encodes and model loads run on threads so the loop never blocks; an `asyncio.Future` bridges the two worlds.
- **The queue decouples them.** `queue.Queue` is the only object shared between the loop and the workers; the “free worker grabs next job” policy is implicit load-balancing.
- **The pool is statically sized.** Worker count is decided once at boot from a peak-memory measurement and a fraction of free VRAM, then frozen for the process lifetime.
- **CPU mode degrades gracefully.** Without CUDA the pool is skipped entirely; a single shared model behind an `asyncio.Lock` plus `run_in_executor` keeps the loop responsive even though encodes are serialised.